

Labo maison : Active Directory & Sécurité

Rapport de projet, Phase 2 (SOC / SIEM avec Wazuh)

Antikat Olouchessi, Polytechnique Montréal, certificat en cybersécurité

1. Contexte et objectif

Après avoir monté l'environnement Active Directory de la Phase 1 (contrôleur de domaine, postes clients, RBAC, GPO), je voulais ajouter un volet surveillance de sécurité, le côté SOC. J'ai installé le SIEM open-source Wazuh pour ça. Le but : détecter des événements de sécurité, analyser des alertes et écrire un rapport d'investigation, exactement ce qu'un analyste SOC niveau 1 fait au quotidien.

Une fois Wazuh en marche, j'ai installé un agent sur DC01 et un autre sur Client01. Ensuite j'ai simulé une attaque simple (des tentatives de connexion échouées répétées) pour voir si le système la détectait, puis j'ai analysé l'alerte générée pour produire mon rapport d'investigation.

2. Choix d'architecture : Docker sur mon Mac plutôt qu'une VM Azure

Au départ, je voulais déployer Wazuh sur une nouvelle VM Ubuntu, dans le même réseau Azure que DC01 et Client01. J'ai abandonné cette idée après plusieurs essais : mon abonnement Azure refusait toutes les tailles de VM que j'essayais (B2s, D2s_v7, et d'autres), pour des raisons de quota régional et d'incompatibilité de contrôleur de disque. J'ai eu beau changer la zone de disponibilité, le type de sécurité, la génération de l'image, rien n'y faisait.

Plutôt que de perdre du temps à contourner les limites de mon abonnement gratuit, j'ai choisi d'installer Wazuh en local avec Docker, sur mon Mac (Apple Silicon M4), celui que j'utilise déjà pour Kali Linux. Ça reste une approche valide pour un labo personnel, et ça me permettait de rester concentrée sur l'objectif principal (apprendre Wazuh), pas sur du dépannage d'infrastructure cloud.

3. Installation de Wazuh via Docker

3.1 Installation de Docker Desktop

J'ai installé Docker Desktop sur mon Mac (version Apple Silicon). Une fois l'appli lancée et le moteur Docker démarré, j'ai confirmé l'installation avec la commande `docker --version` dans le Terminal (Docker version 29.8.0).

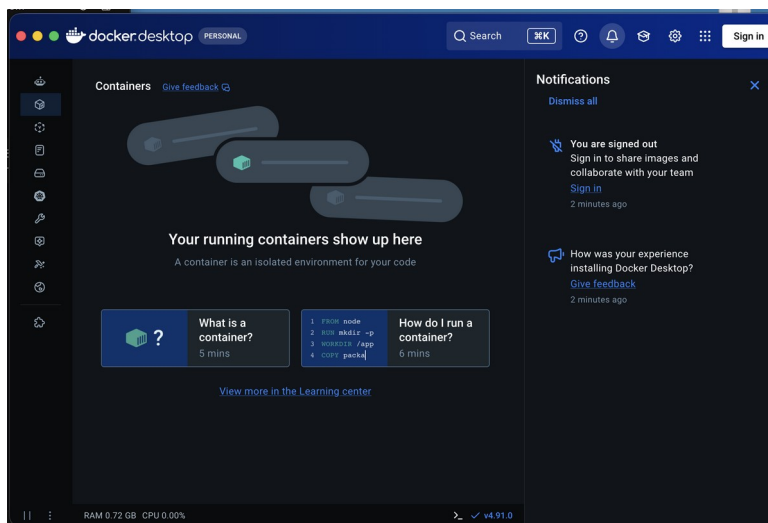


Figure 1. Docker Desktop installé et démarré sur le Mac (moteur actif, version 4.91.0).

Cette capture confirme que le moteur Docker tourne correctement avant même de toucher à Wazuh. C'est une vérification de base, mais essentielle : sans un moteur Docker actif, aucune des commandes docker compose suivantes n'aurait fonctionné.

3.2 Clonage du dépôt Wazuh officiel

J'ai cloné le dépôt officiel wazuh-docker (version v4.14.7, la version stable au moment où j'ai fait le projet) dans un dossier dédié, puis j'ai utilisé le répertoire single-node pour la suite, une configuration Docker Compose déjà prête pour un déploiement Wazuh à un seul nœud (indexeur, serveur et tableau de bord).

3.3 Génération des certificats SSL

La première étape technique, c'est de générer les certificats SSL dont Wazuh a besoin (indexeur, Filebeat, tableau de bord). Je suis tombée sur une erreur de permissions connue sur macOS : le script de génération verrouille lui-même le dossier en lecture seule (droits 500/400) une fois les certificats créés, ce qui empêchait la copie finale de deux fichiers (root-ca-manager.pem et sa clé). J'ai confirmé le problème en vérifiant les permissions avec `ls -la`, puis je l'ai réglé en réappliquant temporairement les droits d'écriture, en copiant le certificat manquant à partir du certificat racine déjà généré, et en remettant le dossier en lecture seule après coup.

```
single-node -- zsh -- 80x24

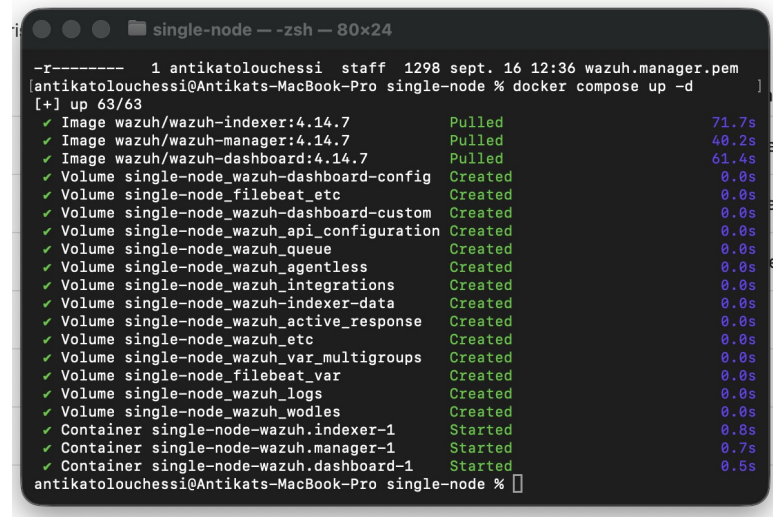
_certs/root-ca.pem config/wazuh_indexer_ssl_certs/root-ca-manager.pem
antikatolouchessi@Antikats-MacBook-Pro single-node % chmod 400 config/wazuh_indexer_ssl_certs/root-ca-manager.pem
antikatolouchessi@Antikats-MacBook-Pro single-node % ls -la config/wazuh_indexer_ssl_certs
total 88
dr-x----- 13 antikatolouchessi  staff  416 sept. 16 12:46 .
drwxr-xr-x  7 antikatolouchessi  staff  224 sept. 15 17:45 ..
-r-----  1 antikatolouchessi  staff  1704 sept. 16 12:36 admin-key.pem
-r-----  1 antikatolouchessi  staff  1220 sept. 16 12:36 admin.pem
-r-----  1 antikatolouchessi  staff  1204 sept. 16 12:46 root-ca-manager.pem
-r-----  1 antikatolouchessi  staff  1704 sept. 16 12:36 root-ca.key
-r-----  1 antikatolouchessi  staff  1204 sept. 16 12:36 root-ca.pem
-r-----  1 antikatolouchessi  staff  1704 sept. 16 12:36 wazuh.dashboard-key.pem
-r-----  1 antikatolouchessi  staff  1302 sept. 16 12:36 wazuh.dashboard.pem
-r-----  1 antikatolouchessi  staff  1704 sept. 16 12:36 wazuh.indexer-key.pem
-r-----  1 antikatolouchessi  staff  1298 sept. 16 12:36 wazuh.indexer.pem
-r-----  1 antikatolouchessi  staff  1708 sept. 16 12:36 wazuh.manager-key.pem
-r-----  1 antikatolouchessi  staff  1298 sept. 16 12:36 wazuh.manager.pem
antikatolouchessi@Antikats-MacBook-Pro single-node %
```

Figure 2. Tous les certificats SSL générés avec succès, permissions restreintes en lecture seule (dossier 500, fichiers 400).

Le fait que les permissions soient revenues à 500/400 confirme que j'ai bien remis le dossier en lecture seule après ma correction manuelle. C'est important : ces certificats servent à chiffrer les communications entre l'indexeur, le serveur et le tableau de bord Wazuh, alors leur intégrité compte.

3.4 Démarrage de la pile Wazuh

Une fois les certificats en place, la commande docker compose up -d a téléchargé les trois images Wazuh (indexeur, serveur/manager, tableau de bord) et démarré les conteneurs correspondants.



```
-r----- 1 antikatolouchessi staff 1298 sept. 16 12:36 wazuh.manager.pem
antikatolouchessi@Antikats-MacBook-Pro single-node % docker compose up -d
[+] up 63/63
✓ Image wazuh/wazuh-indexer:4.14.7 Pulled 71.7s
✓ Image wazuh/wazuh-manager:4.14.7 Pulled 40.2s
✓ Image wazuh/wazuh-dashboard:4.14.7 Pulled 61.4s
✓ Volume single-node_wazuh-dashboard-config Created 0.0s
✓ Volume single-node_filebeat_etc Created 0.0s
✓ Volume single-node_wazuh-dashboard-custom Created 0.0s
✓ Volume single-node_wazuh_api_configuration Created 0.0s
✓ Volume single-node_wazuh_queue Created 0.0s
✓ Volume single-node_wazuh_agentless Created 0.0s
✓ Volume single-node_wazuh_integrations Created 0.0s
✓ Volume single-node_wazuh-indexer-data Created 0.0s
✓ Volume single-node_wazuh_active_response Created 0.0s
✓ Volume single-node_wazuh_etc Created 0.0s
✓ Volume single-node_wazuh_var_multigroups Created 0.0s
✓ Volume single-node_filebeat_var Created 0.0s
✓ Volume single-node_wazuh_logs Created 0.0s
✓ Volume single-node_wazuh_wodles Created 0.0s
✓ Container single-node-wazuh.indexer-1 Started 0.8s
✓ Container single-node-wazuh.manager-1 Started 0.7s
✓ Container single-node-wazuh.dashboard-1 Started 0.5s
antikatolouchessi@Antikats-MacBook-Pro single-node %
```

Figure 3. Les trois images Wazuh sont téléchargées et les conteneurs démarrent sans erreur.

On voit ici les trois services (wazuh.indexer, wazuh.manager, wazuh.dashboard) démarrer l'un après l'autre. Aucune erreur dans les logs à ce stade, ce qui veut dire que les certificats générés plus tôt sont valides et bien reconnus par chaque composant.

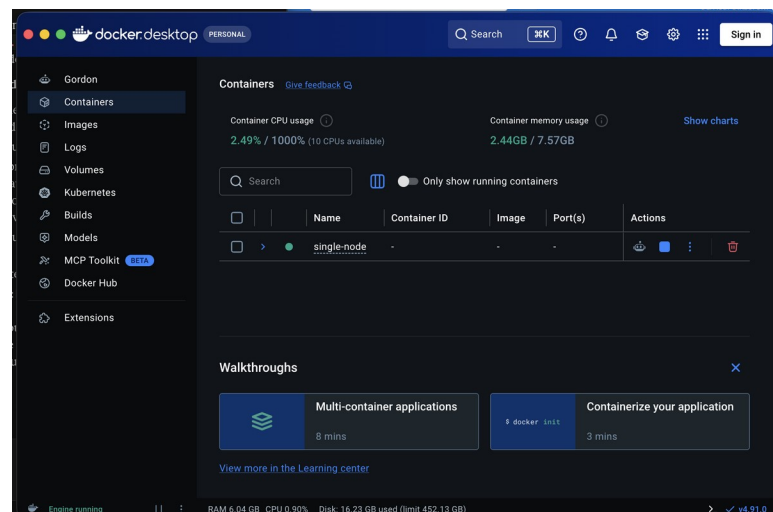


Figure 4. Le conteneur single-node est actif dans Docker Desktop (statut vert).

Le statut vert dans Docker Desktop confirme que les trois conteneurs tournent en continu, pas juste le temps d'un démarrage suivi d'un crash. C'est ce conteneur qui héberge toute la pile Wazuh pour le reste du labo.

4. Accès au tableau de bord Wazuh

Le tableau de bord Wazuh est accessible en local à <https://localhost>. Le navigateur affiche un avertissement de sécurité, mais c'est normal ici : le certificat est auto-signé (généré par Wazuh lui-même), pas délivré par une vraie autorité de certification. Rien d'inquiétant pour un labo local.

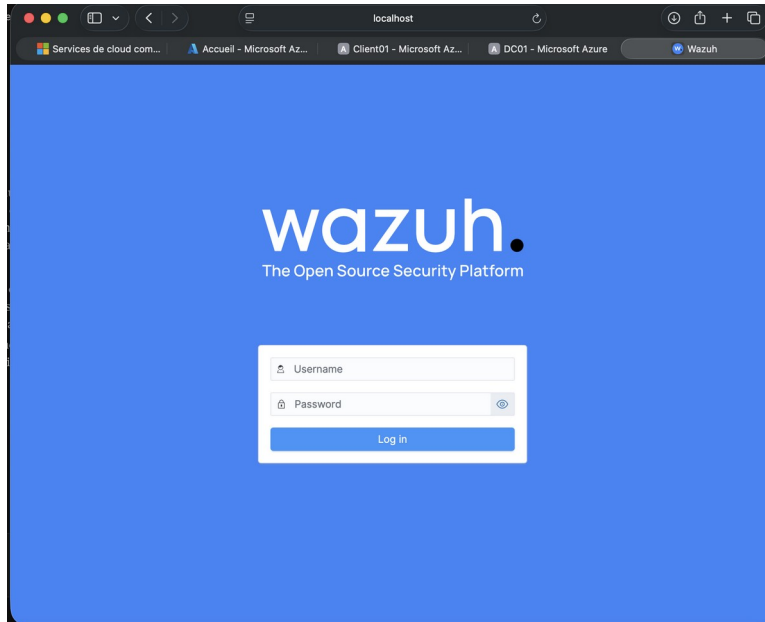


Figure 5. Écran de connexion du tableau de bord Wazuh.

Ce simple écran de connexion confirme que le serveur web du tableau de bord répond, et donc que toute la pile (indexeur, serveur, dashboard) communique correctement entre elle.

Je me suis connectée avec les identifiants par défaut (admin / SecretPassword), un point à corriger avant de considérer le labo terminé, ces identifiants par défaut étant un risque de sécurité bien connu.

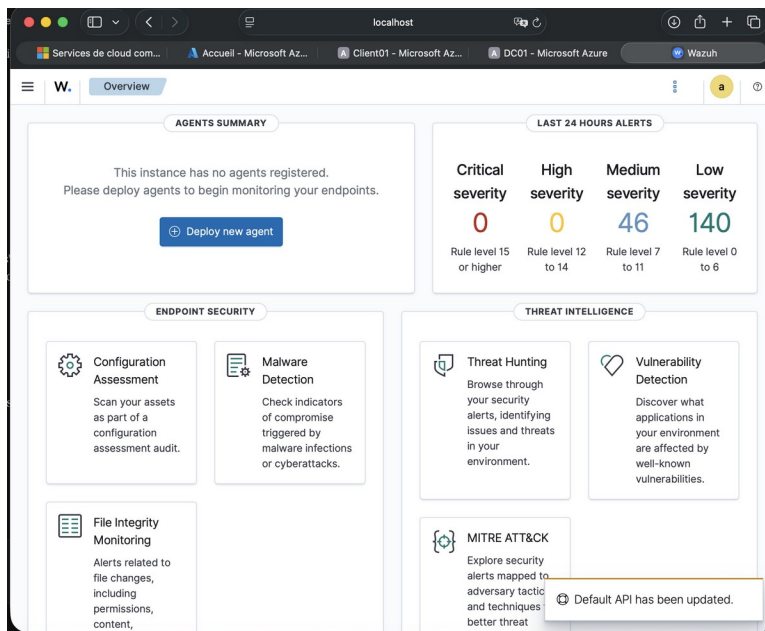


Figure 6. Vue d'ensemble (Overview) du tableau de bord Wazuh : aucun agent n'est encore enregistré à ce stade, mais le serveur, l'indexeur et le tableau de bord fonctionnent.

À ce stade, la vue est vide côté agents, ce qui est normal puisque je n'avais pas encore déployé d'agent sur DC01 ou Client01. Cette capture marque le point de départ avant la partie surveillance proprement dite.

5. Connectivité réseau : Tailscale

DC01 et Client01 sont hébergées dans Azure, alors que mon serveur Wazuh tourne sur mon Mac à la maison. Les agents Wazuh ne pouvaient donc pas atteindre le Mac directement : la plupart des fournisseurs Internet résidentiels bloquent les connexions entrantes, un problème que j'avais déjà croisé avec le test SMB de la Phase 1. Plutôt que de configurer une redirection de ports sur mon routeur, j'ai installé Tailscale (un VPN de type mesh) sur les trois machines (Mac, DC01, Client01), pour leur donner des adresses IP privées communes, joignables entre elles peu importe leur réseau physique.

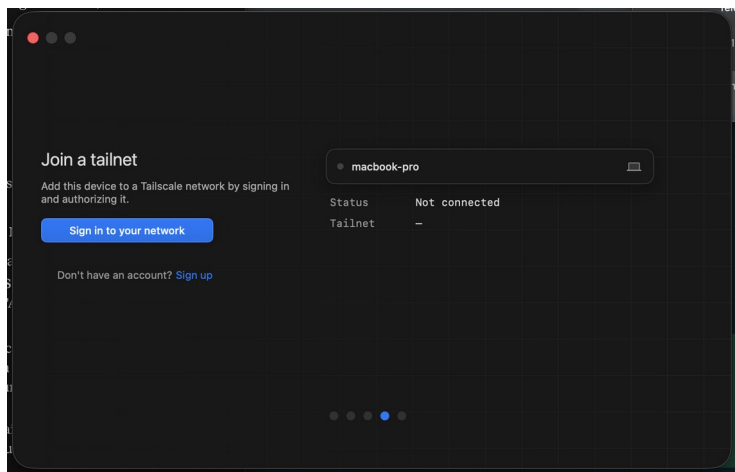


Figure 7. Installation de Tailscale sur le Mac (écran de connexion au réseau Tailscale).

Une fois connecté, le Mac reçoit une adresse Tailscale fixe dans la plage 100.x.x.x. C'est cette adresse, et non l'adresse IP publique du Mac, que les agents Wazuh utilisent ensuite pour parler au serveur.

Sur DC01, je n'avais pas d'identifiants pour l'authentification interactive habituelle (compte GitHub). J'ai donc généré une clé d'authentification depuis mon compte Tailscale déjà connecté sur le Mac, et je l'ai utilisée pour connecter DC01 et Client01 sans passer par un navigateur (tailscale up --authkey=...).

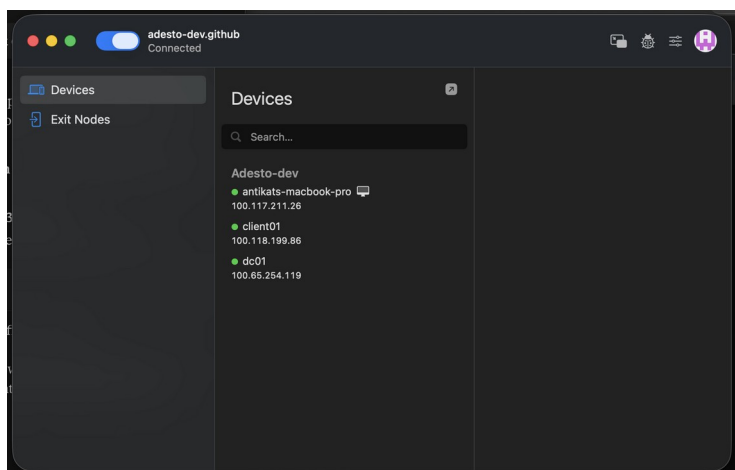


Figure 8. Console Tailscale : les trois appareils (Mac, DC01, Client01) sont connectés au même réseau, chacun avec sa propre adresse Tailscale (plage 100.x.x.x).

Cette vue confirme que les trois machines sont bien sur le même réseau virtuel, malgré le fait qu'elles soient physiquement dans des endroits différents (deux VM Azure aux États-Unis, un Mac chez moi). C'est ce qui rend possible la suite du déploiement.

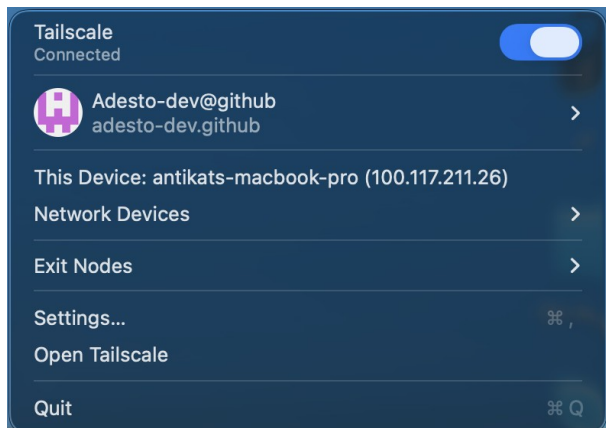


Figure 9. Tailscale connecté sur le Mac, avec son adresse Tailscale (100.117.211.26), l'adresse que j'ai utilisée comme adresse du serveur Wazuh pour les agents.

C'est précisément cette adresse (100.117.211.26) que j'ai entrée dans l'assistant de déploiement des agents Wazuh, à la place d'une adresse IP publique classique.

J'ai testé la connexion (ping) entre Client01 et le Mac via leurs adresses Tailscale, et ça a confirmé que les trois machines pouvaient maintenant communiquer entre elles, malgré leurs réseaux d'origine différents.

6. Déploiement des agents Wazuh

Une fois la connectivité réseau en place, j'ai déployé un agent Wazuh sur chacune des deux machines Windows, avec l'assistant « Deploy new agent » du tableau de bord (système Windows, adresse du serveur = adresse Tailscale du Mac). L'assistant génère une commande PowerShell prête à l'emploi, à exécuter en administrateur sur la machine cible.

6.1 Agent sur DC01

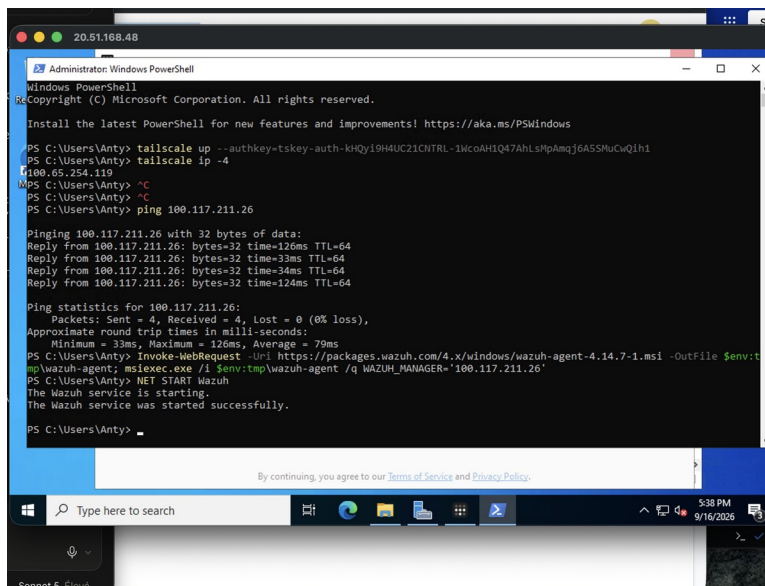


Figure 10. Installation et démarrage réussis de l'agent Wazuh sur DC01 (adresse du serveur : 100.117.211.26, l'adresse Tailscale du Mac).

La commande PowerShell télécharge le programme d'installation de l'agent, l'installe silencieusement, puis démarre le service. Aucune erreur dans la sortie de la commande, donc l'agent devrait apparaître dans le tableau de bord d'un instant à l'autre.

Juste après l'installation, l'agent apparaît dans le tableau de bord avec le statut « pending » (en attente), un état normal et transitoire, le temps que l'agent finisse de se synchroniser avec le serveur.

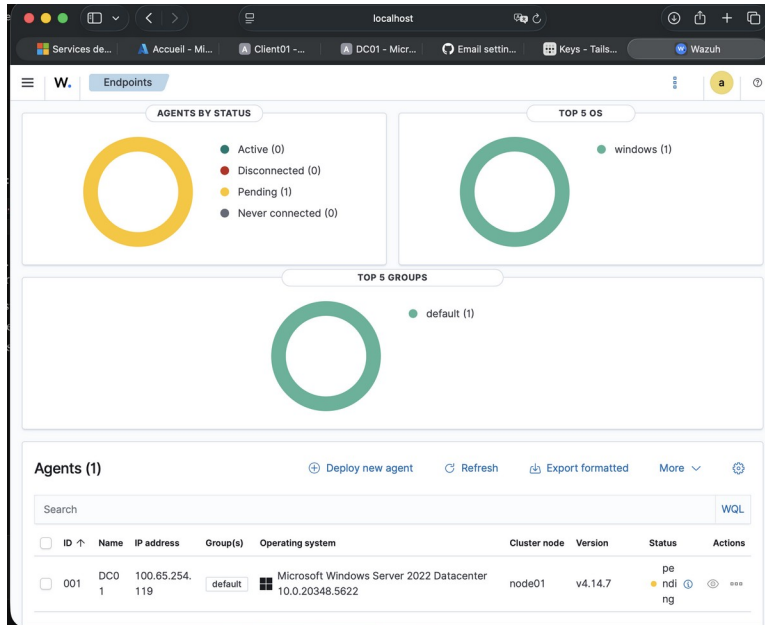


Figure 11. DC01 visible dans la liste des agents, statut « pending » juste après l'installation.

Ce statut « pending » veut dire que le serveur Wazuh a bien reçu une première connexion de l'agent, mais que la synchronisation complète (clés, configuration) n'est pas encore terminée.

Après un court délai et une actualisation de la page, le statut est passé à « Active », ce qui confirme que l'agent communique bien avec le serveur Wazuh.

6.2 Agent sur Client01

J'ai répété la même procédure pour Client01 (même adresse de serveur Tailscale), avec un nom d'agent différent pour bien le distinguer dans le tableau de bord.

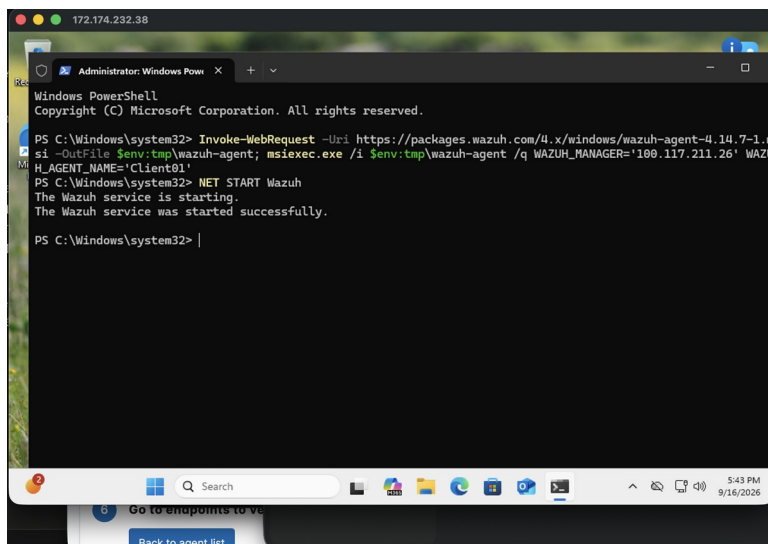


Figure 12. Installation et démarrage réussis de l'agent Wazuh sur Client01.

Même procédure, même résultat : l'agent s'installe et démarre sans erreur. Le fait que ça fonctionne du premier coup sur la deuxième machine confirme que la méthode (adresse Tailscale + assistant de déploiement) est fiable et reproductible.

6.3 Résultat : les deux agents actifs

Les deux agents (DC01 et Client01) apparaissent maintenant avec le statut « Active » dans le tableau de bord Wazuh, le SIEM surveille en temps réel les deux machines Windows de mon labo.

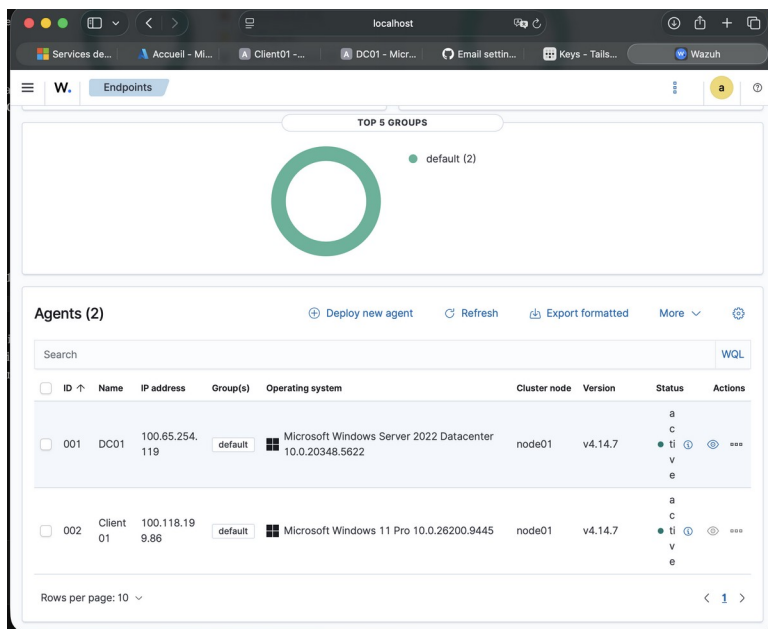


Figure 13. Les deux agents (DC01 et Client01) sont actifs dans le tableau de bord Wazuh.

C'est le résultat clé de cette section : mon SIEM reçoit désormais les journaux d'événements des deux machines en temps réel. Sans cette étape, je n'aurais rien pu détecter dans la section suivante.

7. Simulation d'attaque et investigation

J'ai simulé volontairement une attaque par force brute sur Client01 (six tentatives de connexion avec un mauvais mot de passe). Wazuh a détecté et journalisé les six tentatives (règle 60122, Logon Failure, niveau 5) en quelques secondes. J'ai fait l'analyse complète de cet incident (chronologie, détails techniques, évaluation du risque et recommandations) dans un rapport séparé : Rapport-Investigation-Force-Brute-Client01.docx.

8. Prochaines étapes

- Changer le mot de passe par défaut du tableau de bord Wazuh (admin / SecretPassword).
- Mettre en place une règle de corrélation Wazuh pour augmenter la sévérité des échecs d'authentification répétés (un constat de mon rapport d'investigation).
- Ajouter une stratégie de verrouillage de compte (Account Lockout Policy) via GPO.
- Révoquer les clés d'authentification Tailscale temporaires une fois le labo terminé.